

Week 20: Graded Mini Project

Ornativa Jewels — Retrieval-Based Chatbot (Langflow or local Flowise)

Weekly Outcomes Addressed

- Explain the role of low-code/no-code tools in agentic AI.
- Demonstrate building pipelines with Langflow or Flowise.
- Compare the pros and cons of low-code versus code-based approaches.
- Design simple multi-tool pipelines using visual builders.

Introduction

This Week 20 Graded Mini Project asks you to build a **visual-pipeline jewellery retail chatbot** for a fictional brand, **Ornativa Jewels** (Hyderabad, India), using **Langflow** or a **self-hosted (local) Flowise instance**. The chatbot must answer customer questions by reading and using the product catalogue provided in **Jewellery Details.pdf**. The goal is to practise building a retrieval-based chatbot using a **visual pipeline** (document loading, chunking, embeddings, retrieval, prompting, memory, and testing) without relying on hard-coded answers.

Where to Find the Tasks

The complete brief and the provided catalogue file (*Jewellery Details.pdf*) are available under **Week 20 – Graded Mini Project** on Vocareum. Follow the steps below to build your chatbot and prepare your submission.

Important

This assignment must be completed in **Langflow** (desktop app or local install) or in a **self-hosted (local) Flowise** instance. Do **not** use Flowise Cloud (cloud.flowiseai.com) — new registrations are closed. It **cannot** be completed inside the Vocareum notebook environment.

Tool Setup — Choose One (both are fully approved)

Option A — Langflow (recommended)

A visual, drag-and-drop RAG builder with the same component set as Flowise. Install any one of the following:

Desktop app (easiest, no coding): download Langflow Desktop for Windows or macOS and launch it.

Python (3.10–3.13):

```
pip install uv
uv pip install langflow
langflow run      → open  http://127.0.0.1:7860
```

Docker: `docker run -it -p 7860:7860 langflowai/langflow:latest → http://127.0.0.1:7860`

Option B — Flowise (local / self-hosted)

Run the original tool on your own machine — no cloud account needed:

Node.js (npx): `npx flowise start → open http://localhost:3000`

Docker: `docker run -d -p 3000:3000 flowiseai/flowise → http://localhost:3000`

Either tool works with the same course **OpenAI API key**. Keep the key private — never paste it into screenshots, chats, or shared files. Prefer environment variables or the tool's secure key field.

Project Objective

Your objective is to create and test a **Jewellery Chatbot** that:

- answers customer questions using the catalogue content provided in **Jewellery Details.pdf**,
- supports follow-up questions using short-term conversation context (memory),
- remains polite, concise, and factual in tone,
- and suggests a relevant alternative when an item is out of stock (based on the catalogue).

Tasks

Task 1 — Data Setup

1. Load the provided file **Jewellery Details.pdf** into your tool (*Langflow: File component; Flowise: Document Loader (PDF File) node*).
2. Chunk the content with a text splitter (*Langflow: Split Text; Flowise: Recursive Character Text Splitter*). Suggested chunk size ≈ 1000 , overlap ≈ 200 .
3. Create embeddings (*Langflow: OpenAI Embeddings; Flowise: Embeddings node*).
4. Store the embeddings in a vector store (*Langflow: Chroma DB – local, or built-in Vector Store; Flowise: Vector Store node — Chroma, FAISS, or in-memory*).

Deliverable: A working flow that successfully ingests and indexes the jewellery catalogue.

Task 2 — Create the Query Chain

1. Add a chat model (*Langflow: OpenAI / Language Model; Flowise: Chat Model*) — e.g., OpenAI GPT-4o-mini.
2. Create a prompt with the instruction (*Langflow: Prompt; Flowise: Prompt Template*):
“You are a jewellery store assistant. Use the catalogue data to answer questions accurately.”
3. Connect the components to form a retrieval-based pipeline (**retriever → Prompt → Language Model**).
Tip: in Langflow, start from the built-in **Vector Store RAG** template to wire most of this automatically.
4. Add **Chat Input** and **Chat Output** components/nodes to complete the conversation flow.

Deliverable: A working query-response flow that can answer questions such as:

- “List all available diamond items.”
- “What is the price of the Pearl Necklace?”
- “Which products are made of 22K gold?”

Task 3 — Add Conversational Memory

1. Insert a memory component between the input and model nodes (*Langflow: Memory / Chat Memory; Flowise: Buffer Memory*).
2. Ensure the chatbot can remember the previous query context. Example:

User: Show me diamond products.

Bot: Classic Diamond Ring, Diamond Stud Earrings, and Lotus Pendant are available.

User: What's the price of the ring?

3. Test multiple turns of conversation to confirm memory retention.

Deliverable: A chatbot that maintains short-term conversation context.

Task 4 — Customisation and Testing

1. Customise the system prompt to reflect the brand's tone, e.g.:
“You are Ornativa's virtual jewellery expert. Be polite, concise, and factual.”
2. Test at least **five** queries, including price, material, category, and stock-related questions.
3. If an item is out of stock, configure the chatbot to suggest an alternative product. Example:
“Ruby Solitaire Ring is currently out of stock. You may like the Classic Diamond Ring instead.”

Deliverable: Evidence of testing (queries + responses) and out-of-stock handling with an appropriate alternative suggestion (based on the catalogue).

Deliverables

Mandatory

- **Exported flow (JSON)** — *Langflow: Flow menu → Export; Flowise: Export Chatflow* — or a clearly captured set of screenshots showing the full component/node pipeline and connections.
- **Test evidence:** at least **five** user queries with the chatbot's responses (screenshots or copied logs).
- **Short write-up** (README or PDF) describing: your component/node choices, chunking approach, prompt design, memory setup, and how you handled out-of-stock suggestions.

Constraints

- Do not hard-code answers.
- Do not ignore the provided catalogue; responses must be driven by retrieval from **Jewellery Details.pdf**.
- Do not paste API keys into screenshots or shared files.
- Do not use Flowise Cloud (cloud.flowiseai.com) — new registrations are closed. Use Langflow or a local Flowise install.

Practice Guidelines (for Graded Mini Project)

This graded mini project will not be completed in Vocareum. You will build a **retrieval-based conversational assistant** using **Langflow** or a **self-hosted Flowise instance** with an LLM provider (for example, the **OpenAI API**), focusing on document ingestion, retrieval quality, grounded prompting, memory, and testing.

Using Chat Support for This Assignment (inside Vocareum)

- Open your practice notebook in **Vocareum**. Use the left sidebar to open the **Assistant** panel for clarifications, troubleshooting, or prompt-writing support.
- Monitor your **GenAI Allowance/Budget** in the Assistant panel and plan prompts accordingly. If you exhaust your allowance for a task, contact the Emeritus support team.
- **API key access:** your course-specific API key and usage notes are provided via the guide and Vocareum instructions. Follow those steps to locate your key when needed.

Security warning

Keep your API key *private*. Do not paste it into screenshots, chats, notebooks you share, or public repos. Prefer environment variables or secure key fields in tools; never hard-code keys in shared files. If you suspect exposure, rotate the key immediately per the guide.

Note: You may use Vocareum to access course guidance and API key instructions if needed. However, the chatbot build for this graded mini project must be completed in **Langflow** or a **self-hosted Flowise instance** and cannot be executed inside Vocareum.

Submission Guidelines

Submit **one zip file** containing the following:

- **Exported flow (JSON)** or full workflow screenshots.
- **Test evidence** (at least five queries with responses).
- **README** (or short PDF) explaining your flow, prompt, memory choice, and out-of-stock handling.

Name your file as: **Week 20_Graded Mini Project_[Your Name].zip**

This is a required graded activity and counts towards programme completion. Ensure your chatbot retrieves from the provided catalogue, maintains short-term context where required, and includes clear testing evidence. Your submission should make your design choices easy to evaluate.

Points	100
Submitting	A file upload
File Types	pdf, zip, ipynb, and docx
Allowed Attempts	2

Week 20: Graded Mini Project — Rubric

Applies identically to Langflow and local-Flowise submissions. Total: 100 points.

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Pts
Visual Pipeline Design (Langflow / local Flowise) - Component/node configuration + flow connectivity (Loader/File, Splitter, Embeddings, Vector Store, Retriever, Prompt, LLM, Memory) - Parameter + prompt configuration (brand tone, context grounding, fallback logic incl. out-of-stock) - Labelled screenshots	30 to >28.0 · Excellent All required components are connected correctly with clear logic and strong parameter settings. Prompt includes brand tone, grounded use of catalogue context, and clear fallback logic (e.g., out-of-stock response). Documentation includes labelled screenshots that make the flow easy to follow.	28 to >22.0 · Good Workflow is functional with mostly correct setup and parameters. Minor issues in flow wiring, parameter choices, or prompt refinement; documentation is present but could be clearer.	22 to >16.0 · Satisfactory Core workflow exists, but one or more important elements are weak or missing (e.g., memory wiring, incomplete retriever chain, basic prompt with omissions). Flow may work inconsistently. Documentation is limited.	16 to >0 · Needs Improvement Incomplete or incorrect setup; workflow is not functional or does not reflect the intended retrieval-based pipeline. Prompt/parameters are missing or incorrect; documentation is missing or unclear.	30 pts
Data Handling and Vector Store Setup - PDF ingestion + splitting (chunking strategy + validation via Playground/inspector/logs) - Embeddings + vector storage (Chroma / FAISS / local store) - Retrieval verified from the store	20 to >18.0 · Excellent PDF is loaded correctly and split cleanly with a sensible chunking strategy, confirmed via the Playground/logs (Langflow) or the Flowise inspector/logs. Embeddings are created and stored successfully, and retrieval from the vector store is verified and reliable.	18 to >14.0 · Good PDF processing and vector storage work overall, with minor issues (e.g., chunk size/overlap not ideal or occasional retrieval mismatches).	14 to >10.0 · Satisfactory Document loads, but chunking has noticeable issues or missing content. The vector store exists but is incomplete or inconsistent, leading to weaker retrieval quality.	10 to >0 · Needs Improvement PDF not processed correctly or not visible to the retriever. Embeddings/vector store setup fails or is not usable.	20 pts
Conversational Logic and Memory - Memory integration (correct placement + wiring) - Multi-turn continuity (2–3 turns) - Follow-ups handled logically	20 to >18.0 · Excellent Memory component is correctly connected and the chatbot retains context across 2–3 turns with clear continuity. Multi-turn follow-ups are handled coherently and consistently.	18 to >14.0 · Good Memory works for short sequences but may occasionally reset context. Multi-turn interaction is present but sometimes less coherent or less accurate.	14 to >10.0 · Satisfactory Memory is partially functional or positioned incorrectly; follow-ups are handled inconsistently. Context continuity is weak.	10 to >0 · Needs Improvement No working memory; chatbot behaves as single-turn with no contextual continuity.	20 pts
Testing and Response Quality - Accuracy of responses (price/material/availability) - Retrieval relevance (avoids irrelevant pulls/hallucinations) - Out-of-stock handling + alternative suggestions	20 to >18.0 · Excellent Responses are factually accurate across test queries, based on retrieved catalogue content. Out-of-stock cases are handled gracefully with contextually relevant alternatives.	18 to >14.0 · Good Responses are mostly correct with one or two minor factual errors or occasional retrieval mismatch. Out-of-stock suggestions are present but may lack clear rationale.	14 to >10.0 · Satisfactory Several inaccuracies or irrelevant retrievals. Out-of-stock handling is incomplete (mentions stock status but does not provide a solid alternative).	10 to >0 · Needs Improvement Responses are frequently incorrect or hallucinated. Out-of-stock queries return empty, wrong, or unsupported results.	20 pts
Documentation and Presentation - Workflow documentation (screenshots + component descriptions + explanation of stages) - Test query summary (≥5 queries + responses + brief reflections)	10 to >9.0 · Excellent Clear screenshots and explanations for each stage of the flow. Includes at least five test queries with corresponding responses and brief reflections aligned to instructions.	9 to >7.0 · Good Screenshots and test queries are included but explanations or coverage are incomplete in places.	7 to >5.0 · Satisfactory Limited documentation and/or fewer than five complete test queries; context and reasoning are unclear.	5 to >0 · Needs Improvement Documentation and test evidence are missing, unclear, or not aligned to instructions.	10 pts

Total Points: 100